

Instituto Politécnico Nacional

Centro de Investigación en Computación (CIC)

Departamento de diplomados y extensión profesional

Curso de Inteligencia Artificial con Python

Profesor Alan Badillo Salas

Proyectos Finales

Regresión, Clasificación, Redes Neuronales y Agentes Inteligentes

Semana 7. Marzo 7, 2026.

1. Introducción

Los siguientes proyectos demostrarán las habilidades desarrolladas para el curso, selecciona un proyecto y documenta el reporte que será evaluado como parte de la calificación final del curso (*Documento del Proyecto Final*).

1.1. Entregables

El proyecto tendrá un único entregable que consiste en un documento PDF creado en cualquier editor de texto que soporte incrustación de imágenes como Word, Excel, Power Point, Canvas o Latex y que permita la exportación o impresión al formato PDF. También se aceptarán trabajos hechos a mano siempre y cuando sean legibles y estén escaneados en formato PDF no mayor a 20mb.

Las especificaciones del *Documento del Proyecto Final* son:

- **Portada** - Debe contener el nombre del Instituto Politécnico Nacional, el nombre del Centro de Investigación en Computación (CIC) y el Departamento de diplomados y extensión profesional, así como el logo del IPN a la izquierda y del CIC a la derecha en las esquinas superiores. También deberá decir Curso de Inteligencia Artificial con Python y abajo el nombre del profesor, Maestro Alan Badillo Salas y el nombre del alumno (tu nombre). Después pon el nombre del práctica (por ejemplo, Proyecto Final - Regresión). Hasta abajo deberá contener la fecha de la práctica o la fecha en que se realizó en el formato: Entidad del instituto. Mes día, año (por ejemplo, Ciudad de México, Marzo 7, 2026).
- **Introducción** - Explica en un párrafo de qué trata el problema abordado por el proyecto final.
- **Desarrollo** - En el desarrollo deberás explicar la secuencia lógica que se siguió en el proyecto final-
- **Resultados** - En esta sección pega las capturas de pantalla de las gráficas generadas y resultados alcanzados (tablas y celdas de la libreta que contengan métricas de evaluación o resultados de interés).
- **Conclusiones** - En las conclusiones deberás resumir que se hizo en general, para qué sirve y qué aplicaciones actuales y futuras tiene (por ejemplo, ¿Se puede aplicar a tu trabajo actual? ¿Qué tipo de proyecto será interesante hacer ahora?).

1.2. Evaluación

El proyecto final tiene un valor de 100 puntos, de los cuales 60 son suficientes para obtener una nota aprobatoria, si tiene menos de 60 puntos no será aceptado y tendrá que ser corregido.

- **10 puntos** - La portada tiene todos los elementos

- **10 puntos** - El proyecto final contine una Introducción razonable
- **20 puntos** - El proyecto final contine la secuencia lógica de pasos completa y es razonable
- **30 puntos** - El proyecto final contine los resultados solicitados
- **30 puntos** - El proyecto final contine conclusiones razonables

En el proyecto final se considera un mayor peso a los resultados y las conclusiones, por lo que las conclusiones deben ser precisas sobre lo que se entendió en general.

Si lo desea puede hacer más de un proyecto, pero no es necesario hacer el reporte, aunque es altamente recomendable para anexarlo en su repositorio de Github, también sería bueno hacer una presentación con las capturas de pantalla entre 5 y 10 diapositivas máximo la cual servirá de portada del repositorio para quién revise su trabajo.

2. Proyecto I - Regresión

Este proyecto tiene como objetivo dominar el análisis del aprendizaje supervisado en respuestas continuas usando el método de regresión.

Procedimiento

1. Selecciona un conjunto de datos del que identifiques una respuesta numérica, por ejemplo, el precio de una casa o la calificación final de un estudiante.
2. Detemrina las características informativas que podrían ayudar a predecir la respuesta, ignorando aquellas que expliquen directamente la respuesta o sean derivas de la misma respuesta, por ejemplo, si es el precio de la casa ignora una posible respuesta que sea el impuesto sobre el valor ya que sería el mismo precio por algún escalar que como el 16 %.
3. Construye cada característica de forma numérica, si la característica es categórica separa cada clase como una variable binaria independiente, excepto para la clase base (la que tiene más observaciones). Si la característica es categórica pero es ordinal (las categorías se pueden ver como ponderaciones del menor al mayor), mejor trata la variable como numérica transformando las categorías a ponderaciones con 1, 2, 3, ..., o con valores negativos y positivos si es mejor, por ejemplo, si la característica es tipo de dieta del estudiante, puedes usar -1 para una dieta mala, 0 para una dieta regular y 1 para una dieta buena. Cada característica codificada debería quedar como $x_1, x_2, \dots, x_k$.
4. Construye el vector de respuesta para la característica numérica de respuesta como y .
5. Construye una gráfica adecuada para cada característica para ver cómo se comporta la respuesta numérica respecto a la característica informativa, usa *cat-num* si la característica informativa es categórica (gráfica de caja para la respuesta segmentada por la característica informativa), usa *num-num* si la característica (gráfica de puntos con densidad o *jointplot*).
6. Construye la matriz de características (cubo de datos) X con todas las características informativas.
7. Separa la matriz de características y el vector de respuesta en las matrices de características para entrenamiento X_{train} y pruebas X_{test} y los vectores de respuesta para entrenamiento y_{train} y pruebas y_{test} . Determina cuántas observaciones sería bueno dejar según la cantidad total de observaciones disponibles (o usa el 20 %). Usa una semilla aleatoria para partir los datos, por ejemplo, 123.
8. Crea al menos un modelo de regresión lineal.

9. Ajusta el modelo a la matriz de características de entrenamiento y el vector de respuesta de entrenamiento.
10. Predice el vector de respuesta $\hat{y}$ para la matriz de características X .
11. Predice el vector de respuesta $\hat{y}_{train}$ para la matriz de características X_{train} .
12. Predice el vector de respuesta $\hat{y}_{train}$ para la matriz de características X_{test} .
13. Grafica la linealidad entre la respuesta y y la respuesta de predicción $\hat{y}$ (usa *regplot*).
14. Grafica la linealidad entre la respuesta y_{train} y la respuesta de predicción $\hat{y}_{train}$ (usa *regplot*).
15. Grafica la linealidad entre la respuesta y_{test} y la respuesta de predicción $\hat{y}_{test}$ (usa *regplot*).
16. Calcula el vector de error $e = y - \hat{y}$
17. Calcula el vector de error $e_{train} = y_{train} - \hat{y}_{train}$
18. Calcula el vector de error $e_{test} = y_{test} - \hat{y}_{test}$
19. Grafica la densidad del error e para observar su normalidad (usa *kdeplot*).
20. Grafica la densidad del error e_{train} para observar su normalidad (usa *kdeplot*).
21. Grafica la densidad del error e_{test} para observar su normalidad (usa *kdeplot*).
22. Calcula las métricas $SSE, MSE, RMSE, MAE$ para el error e .
23. Calcula las métricas $SSE_{train}, MSE_{train}, RMSE_{train}, MAE_{train}$ para el error e_{train} .
24. Calcula las métricas $SSE_{test}, MSE_{test}, RMSE_{test}, MAE_{test}$ para el error e_{test} .
25. Reporta las métricas en una tabla la comparación de las métricas, usa las columnas *Total, Train, Test* y las filas *SSE, MSE, RMSE, MAE*.
26. **(Opcional)** Repite la regresión usando un método de regresión no lineal, por ejemplo, un soporte vectorial con *kernel* polinomial de grado 2, 3 o 5, o un *kernel* de tipo *rbf*. O algún otro método de regresión que estudies como árboles de decisión o bosques aleatorios para regresión. Justifica brevemente su comportamiento. Genera nuevamente las gráficas y el reporte de las métricas.

27. **(Opcional)** Normaliza la matriz de características X_s mediante el *StandardScaler* y repite el método regresión lineal y el método de regresión no lineal que elegiste.
28. **(Opcional)** Extrae el interceptor β_0 y los coeficientes de regresión $\beta_1, \beta_2, \dots, \beta_k$ para el método de regresión lineal con la matriz de características normalizada X_s .
29. **(Opcional)** Reporta los coeficientes de regresión (betas) en una tabla y observa cuales son muy positivos, muy negativos o cercanos a cero (características informativa con influencia lineal casi nula).
30. **(Opcional)** Estudia el factor de inflación de la varianza **VIF** y calcula para cada característica la redundancia o multicolinealidad hacia la respuesta para el método de regresión lineal.
31. **(Opcional)** Reporta en una tabla el **VIF** de cada característica informativa.
32. **(Opcional)** Repite el modelo normalizado de regresión lineal y no lineal quitando las características poco informativas (determinadas por los coeficientes de regresión o betas cercanas a cero).
33. **(Opcional)** Calcula las métricas adicionales de R^2 y $R^2_{ajustada}$ sobre el modelo con todas las características, sobre el modelo sin las características poco informativas (determinadas por los coeficientes de regresión o betas cercanas a cero) y sobre el modelo sin las características de mayor inflación de la varianza.
34. **(Opcional)** Extiende características cruzadas bajo un planteamiento objetivo conociendo el conjunto de datos, por ejemplo, para los estudiantes se podrían construir variables que crucen la información entre las horas de estudio y las horas de ocio (redes sociales y netflix) o en el conjunto de casas variables que crucen el área de construcción por la calidad de la casa. Repite los modelos y métricas bajo aportando estas nuevas características.

Análisis

1. ¿Cuántas características informarán la respuesta?
2. ¿Qué características podrían informar mejor a la respuesta visualmente según las gráficas?
3. ¿Qué tan lineal son las respuestas y las predicciones?
4. ¿Qué tan normales son los errores?
5. ¿Cómo cambian las métricas entre el entrenamiento, las pruebas y el total?
6. ¿Cómo determinarías si hay sobreajuste comparando las métricas de entrenamiento y las de pruebas?

7. **(Opcional)** ¿Cómo mejoran o empeoran las predicciones del modelo lineal y el no lineal?
8. **(Opcional)** ¿Cómo mejoran o empeoran las predicciones del modelo lineal y el no lineal estandarizado?
9. **(Opcional)** ¿Qué representa una beta muy positiva?
10. **(Opcional)** ¿Qué representa una beta muy negativa?
11. **(Opcional)** ¿Qué representa una beta cercana a cero?
12. **(Opcional)** ¿Qué variables influyen poco linealmente a la respuesta?
13. **(Opcional)** ¿Qué variables son altamente colineales?
14. **(Opcional)** ¿Cómo mejora o empeora la predicción sin las características poco informativas linealmente?
15. **(Opcional)** ¿Cómo mejora o empeora la predicción sin las características con alta colinealidad lineal?
16. **(Opcional)** ¿Cómo se interpretan las métricas de R^2 y $R^2_{ajustada}$?
17. **(Opcional)** ¿Cuál concluyes que es el mejor modelo final?
18. **(Opcional)** ¿Sirvió informar sobre las características cruzadas adicionales?

Recomendación para nivel maestría

Un análisis más profundo se puede utilizar validación cruzada, pruebas de normalidad (Shapiro-Wilk o Jarque-Bera), modelos regularizados (Ridge, Lasso, Elastic-Net), calcular la importancia de las características informativas con métodos más avanzados como *permutation importance* o *SHAP values*. Análisis de *sesgo-varianza* entre los diferentes modelos lineales y no lineales para detectar interpretabilidad, complejidad y estabilidad. Transformaciones de las características informativas más avanzadas como transformaciones logarítmicas, polinomios cruzados e interacciones. También se puede alcanzar el diagnóstico de supuestos, regularización, interpretabilidad y la justificación de decisiones.

Con esto se pueden alcanzar proyectos tipo *Stanford ML*, *MIT data science* y programas de econometría aplicada.

3. Proyecto II - Clasificación

Este proyecto tiene como objetivo dominar el análisis del aprendizaje supervisado en respuestas categóricas usando métodos de clasificación.

Procedimiento

1. Selecciona un conjunto de datos del que identifiques una respuesta categórica, por ejemplo, si un estudiante aprueba o reprueba una asignatura o si un paciente tiene o no una enfermedad. La respuesta puede ser binaria o multibinaria.
2. Determina las características informativas que podrían ayudar a predecir la respuesta categórica, ignorando aquellas que expliquen directamente la respuesta o sean derivadas de la misma respuesta.
3. Construye cada característica informativa de forma numérica. Si la característica es categórica separa cada clase como una variable binaria independiente, excepto para la clase base (la que tiene más observaciones). Si la característica es ordinal (las categorías se pueden ordenar de menor a mayor), puedes transformarla en valores numéricos como 1, 2, 3, ... o valores negativos y positivos si es conveniente. Cada característica codificada debería quedar como $x_1, x_2, \dots, x_k$.
4. Construye el vector de respuesta categórica como y .
5. Construye una gráfica adecuada para cada característica informativa para observar cómo se distribuye la respuesta categórica respecto a la característica. Usa *cat-cat* si la característica informativa es categórica (mapa de calor). Usa *num-cat* si la característica es numérica (gráfica de densidad o cajas separados por las clases de la respuesta).
6. Construye la matriz de características (cubo de datos) X con todas las características informativas.
7. Separa la matriz de características y el vector de respuesta en las matrices de características para entrenamiento X_{train} y pruebas X_{test} y los vectores de respuesta para entrenamiento y_{train} y pruebas y_{test} . Determina cuántas observaciones sería bueno dejar según la cantidad total de observaciones disponibles (o usa el 20 %). Usa una semilla aleatoria para partir los datos, por ejemplo, 123.
8. Crea un modelo de clasificación usando la curva logística o un árbol de decisión.
9. Ajusta el modelo a la matriz de características de entrenamiento y el vector de respuesta de entrenamiento.
10. Predice el vector de respuesta categórica $\hat{y}$ para la matriz de características X .

11. Predice el vector de respuesta $\hat{y}_{train}$ para la matriz de características X_{train} .
12. Predice el vector de respuesta $\hat{y}_{test}$ para la matriz de características X_{test} .
13. Construye la matriz de confusión para la predicción total comparando y contra $\hat{y}$.
14. Construye la matriz de confusión para el conjunto de entrenamiento comparando y_{train} contra $\hat{y}_{train}$.
15. Construye la matriz de confusión para el conjunto de pruebas comparando y_{test} contra $\hat{y}_{test}$.
16. Grafica los mapas de calor con las matrices de confusión total, entrenamiento y pruebas.
17. A partir de la matriz de confusión calcula las siguientes métricas para el conjunto total:
 - Exactitud
 - Precisión
 - Sensibilidad
 - Especificidad
 - F1
18. Calcula las mismas métricas para el conjunto de entrenamiento.
19. Calcula las mismas métricas para el conjunto de pruebas.
20. Reporta las métricas en una tabla comparativa usando las columnas Total, Train, Test y las filas Exactitud, Precisión, Sensibilidad, Especificidad, F1.
21. **(Opcional)** Repite el experimento usando otro método de clasificación diferente (por ejemplo soporte vectorial o bosques aleatorios). Justifica brevemente el comportamiento observado y compara las métricas.
22. **(Opcional)** Normaliza la matriz de características X_s mediante el *StandardScaler* y repite el entrenamiento del modelo de clasificación.
23. **(Opcional)** Analiza la importancia de las características informativas usando el modelo utilizado (coeficientes en regresión logística o importancia de variables en árboles de decisión).
24. **(Opcional)** Extiende características cruzadas bajo un planteamiento objetivo conociendo el conjunto de datos. Por ejemplo, en estudiantes se podrían cruzar horas de estudio con horas de ocio (redes sociales y netflix), o en un conjunto médico cruzar edad con hábitos de salud. Repite el modelo utilizando estas nuevas características.

Análisis

1. ¿Cuántas características informativas se utilizaron para predecir la respuesta categórica?
2. ¿Qué características parecen separar mejor las clases según las gráficas?
3. ¿Cómo se interpreta la matriz de confusión obtenida?
4. ¿Cómo cambian las métricas entre entrenamiento, pruebas y total?
5. ¿Cómo determinarías si existe sobreajuste comparando las métricas de entrenamiento y pruebas?
6. ¿Qué representa una precisión alta?
7. ¿Qué representa una sensibilidad alta?
8. ¿Qué representa una especificidad alta?
9. ¿En qué situaciones sería más importante maximizar la sensibilidad?
10. ¿En qué situaciones sería más importante maximizar la especificidad?
11. ¿Cómo se interpreta la métrica F1?
12. **(Opcional)** ¿Cómo cambian las métricas al utilizar otro modelo de clasificación?
13. **(Opcional)** ¿Cómo cambian las métricas al normalizar las características informativas?
14. **(Opcional)** ¿Qué características parecen tener mayor influencia en la clasificación?
15. **(Opcional)** ¿Las características cruzadas mejoraron la clasificación?
16. **(Opcional)** ¿Cuál concluyes que es el mejor modelo final?

Recomendación para nivel maestría

Un análisis más profundo puede incluir el uso de validación cruzada para evaluar la estabilidad del modelo de clasificación mediante múltiples particiones del conjunto de datos. También se pueden analizar curvas ROC y el área bajo la curva (*AUC*) para evaluar el comportamiento del clasificador bajo diferentes umbrales de decisión.

Se pueden aplicar modelos de clasificación más avanzados como bosques aleatorios, *gradient boosting*, máquinas de soporte vectorial con diferentes *kernels* o redes neuronales. También se puede analizar el problema de desbalance de clases mediante técnicas como re-muestreo (*oversampling* o *undersampling*) o ponderación de clases durante el entrenamiento.

Otro análisis relevante consiste en estudiar la importancia de las características informativas mediante métodos más avanzados como *permutation importance* o *SHAP values* para comprender la influencia de cada característica en las decisiones del modelo.

También se puede extender la ingeniería de características informativas mediante interacciones entre variables, transformaciones no lineales o combinaciones que reflejen mejor el fenómeno estudiado. Finalmente, se puede realizar un análisis más profundo del compromiso entre interpretabilidad, complejidad y desempeño del modelo, justificando la selección del clasificador final.

Con esto se pueden alcanzar proyectos tipo *Stanford ML*, *MIT data science* y programas de econometría aplicada.

4. Proyecto III - Redes Neuronales

Este proyecto tiene como objetivo dominar el uso de redes neuronales artificiales para resolver problemas de aprendizaje supervisado tanto en respuestas continuas (regresión) como en respuestas categóricas (clasificación).

Procedimiento

1. Selecciona un conjunto de datos del que identifiques una respuesta que puede ser numérica (problema de regresión) o categórica (problema de clasificación binaria o multibinaria).
2. Determina las características informativas que podrían ayudar a predecir la respuesta, ignorando aquellas que expliquen directamente la respuesta o sean derivadas de la misma respuesta.
3. Construye cada característica informativa de forma numérica. Si la característica es categórica separa cada clase como una variable binaria independiente, excepto para la clase base (la que tiene más observaciones). Si la característica es ordinal puedes transformarla en valores numéricos ordenados.
4. Construye el vector de respuesta como y . Si el problema es de clasificación multibinaria puedes transformar la respuesta en variables binarias independientes para cada clase.
5. Construye una gráfica adecuada para cada característica informativa para observar cómo se comporta la respuesta respecto a la característica (usa los mismos criterios de exploración vistos en los proyectos anteriores).
6. Construye la matriz de características (cubo de datos) X con todas las características informativas.
7. Separa la matriz de características y el vector de respuesta en matrices de entrenamiento X_{train} y pruebas X_{test} y los vectores de respuesta y_{train} y y_{test} . Determina cuántas observaciones sería bueno dejar según la cantidad total de observaciones disponibles (o usa el 20 %). Usa una semilla aleatoria para partir los datos, por ejemplo 123.
8. Construye una red neuronal con capas densas. Utiliza una capa de entrada acorde al número de características informativas.
9. Construye al menos una arquitectura sencilla con pocas neuronas y poca profundidad (por ejemplo una o dos capas densas intermedias). No olvides ajustar en las capas intermedias la activación *relu* y compilar bien el modelo con un optimizador adecuado como *sgd* o *adam*, la función de pérdida *mse* o *mae* para regresión o *binary_crossentropy* para clasificación binaria o *categorical_crossentropy* para clasificación multibinaria.

10. Selecciona la función de activación de la capa de salida según el tipo de problema:
 - Activación *linear* si la respuesta es numérica (regresión)
 - Activación *sigmoid* si la respuesta es categórica binaria
 - Activación *softmax* si la respuesta es categórica multibinaria
11. Ajusta la red neuronal utilizando el conjunto de entrenamiento durante un número adecuado de épocas.
12. Utiliza un tamaño de lote (*batch size*) adecuado para el entrenamiento. Lotes muy pequeños pueden producir fluctuaciones en el aprendizaje mientras que lotes más grandes pueden estabilizar el proceso de optimización.
13. Obtén las predicciones de la red neuronal para el conjunto total $\hat{y}$.
14. Obtén las predicciones para el conjunto de entrenamiento $\hat{y}_{train}$.
15. Obtén las predicciones para el conjunto de pruebas $\hat{y}_{test}$.
16. Si el problema es de regresión calcula la métrica de error absoluto medio (MAE) para el conjunto total, entrenamiento y pruebas.
17. Si el problema es de clasificación construye la matriz de confusión para el conjunto total, entrenamiento y pruebas.
18. Reporta las métricas en una tabla comparativa con las columnas Total, Train, Test.
19. **(Opcional)** Construye una segunda arquitectura con mayor número de neuronas o mayor profundidad de capas densas intermedias..
20. **(Opcional)** Compara el comportamiento de las diferentes arquitecturas de red neuronal y observa cuál obtiene mejores resultados.
21. **(Opcional)** Si el problema es de regresión extiende las métricas como en el Proyecto I (SSE, MSE, RMSE y MAE).
22. **(Opcional)** Si el problema es de clasificación extiende las métricas como en el Proyecto II (exactitud, precisión, sensibilidad, especificidad y F1).
23. **(Opcional)** Experimenta con diferentes números de neuronas en las capas intermedias.
24. **(Opcional)** Experimenta con diferentes profundidades de la red neuronal (mayor número de capas densas).
25. **(Opcional)** Ajusta el número de épocas para observar cómo evoluciona el aprendizaje de la red neuronal.

26. **(Opcional)** Ajusta el tamaño del lote para observar cambios en la estabilidad del entrenamiento.
27. **(Opcional)** Normaliza la matriz de características informativas utilizando *StandardScaler* antes del entrenamiento.
28. **(Opcional)** Grafica la evolución de la función de pérdida durante el entrenamiento para observar el proceso de aprendizaje.

Análisis

1. ¿Cuántas características informativas se utilizaron para entrenar la red neuronal?
2. ¿Qué arquitectura de red neuronal obtuvo mejores resultados?
3. ¿Cómo cambian las métricas entre entrenamiento y pruebas?
4. ¿Se observa sobreajuste en alguna arquitectura de red neuronal?
5. ¿Cómo afecta el número de neuronas al desempeño del modelo?
6. ¿Cómo afecta la profundidad de la red neuronal al desempeño del modelo?
7. ¿Cómo afecta el número de épocas al aprendizaje de la red?
8. ¿Cómo afecta el tamaño del lote a la estabilidad del entrenamiento?
9. **(Opcional)** ¿La normalización de las características informativas mejoró el desempeño del modelo?
10. **(Opcional)** ¿La red neuronal supera o no a los modelos utilizados en los proyectos anteriores?
11. **(Opcional)** ¿Cuál arquitectura final elegirías y por qué?

Recomendación para nivel maestría

Un análisis más profundo puede incluir el uso de validación cruzada para evaluar la estabilidad del modelo bajo diferentes particiones del conjunto de datos. También se pueden explorar arquitecturas más complejas de redes neuronales con mayor profundidad o diferentes configuraciones de neuronas en cada capa.

Se pueden analizar técnicas de regularización como *dropout*, penalizaciones sobre los pesos o detención temprana (*early stopping*) para reducir el sobreajuste. Asimismo, se pueden estudiar diferentes funciones de activación en las capas intermedias y distintos algoritmos de optimización para el entrenamiento de la red.

Otro análisis relevante consiste en estudiar la interpretabilidad del modelo mediante métodos de importancia de características como *permutation importance* o *SHAP values*. También se pueden comparar los resultados obtenidos con redes neuronales frente a los modelos lineales y no lineales desarrollados en los proyectos anteriores.

Finalmente, se puede estudiar el compromiso entre complejidad del modelo, estabilidad del aprendizaje e interpretabilidad de los resultados para justificar la selección de la arquitectura final de la red neuronal.

5. Proyecto IV - Agentes Inteligentes

Este proyecto tiene como objetivo diseñar un agente inteligente capaz de interpretar instrucciones en lenguaje natural y generar respuestas estructuradas o ejecutar tareas específicas mediante el uso de modelos generativos de lenguaje.

El objetivo del proyecto es comprender cómo los modelos de lenguaje pueden actuar como agentes capaces de transformar texto libre en información útil, estructurada o accionable.

Procedimiento

1. Selecciona uno de los siguientes tipos de agente inteligente vistos en el laboratorio:
 - Agente generador de texto
 - Agente con llamadas a función
 - Agente generador de búsquedas estructuradas
 - Agente extractor de información desde documentos
2. Define el problema que resolverá el agente. Describe brevemente el tipo de tarea que el agente realizará a partir de instrucciones escritas por el usuario.
3. Diseña un conjunto de instrucciones o *prompt* que guíe al modelo para realizar la tarea deseada.
4. Diseña el formato de salida esperado del agente. Este formato puede ser texto libre o información estructurada como listas, tablas o estructuras tipo JSON. Utiliza el modelo de *tool functions* para conseguirlo.
5. Construye varios ejemplos de entrada de usuario que representen situaciones reales del problema que el agente debe resolver.
6. Ejecuta el agente con diferentes ejemplos de entrada y observa el comportamiento del modelo.
7. Si el agente utiliza llamadas a función, diseña la estructura de los parámetros que el modelo deberá generar para ejecutar la función correspondiente.
8. Si el agente genera búsquedas estructuradas, diseña los campos que compondrán la consulta (por ejemplo: categoría, precio, marca, características técnicas).
9. Si el agente realiza extracción de información desde documentos, define los campos que se deberán extraer del texto (por ejemplo: nombres de actores, fechas, lugares, conceptos jurídicos u otros elementos relevantes).

10. Evalúa si las respuestas del agente cumplen correctamente con el formato de salida esperado.
11. Realiza varias pruebas con diferentes tipos de entradas de usuario y observa la estabilidad del comportamiento del agente.
12. Reporta ejemplos de entradas de usuario y las respuestas generadas por el agente.
13. **(Opcional)** Ajusta el diseño del *prompt* o las instrucciones del agente para mejorar la precisión de las respuestas generadas.
14. **(Opcional)** Describe las limitaciones observadas en el comportamiento del agente al cambiar el diseño o colocar respuestas incoherentes.
15. **(Opcional)** Crea un mapa de flujos o un esquema que integre varios agentes, una arquitectura de comunicación entre el cliente y el servidor, indicando almacenamiento de respuestas y resultados en la base de datos y llamadas a *APIs* externas de servicios de reservación o de consulta.

Análisis

1. ¿Qué tipo de agente inteligente se implementó?
2. ¿Qué tipo de instrucciones o *prompt* fueron necesarias para guiar al modelo?
3. ¿Qué tan consistentes fueron las respuestas generadas por el agente?
4. ¿El agente logró producir información estructurada correctamente?
5. ¿Qué tipos de errores se observaron en las respuestas generadas?
6. ¿Cómo cambió el comportamiento del agente al modificar el diseño del *prompt*?
7. ¿Qué limitaciones observaste en el modelo para resolver la tarea?
8. ¿En qué contextos reales podría utilizarse el agente desarrollado?
9. **(Opcional)** ¿Cómo podría integrarse este agente dentro de un sistema más grande (por ejemplo una aplicación web, chatbot o sistema de consulta)?
10. **(Opcional)** ¿Cómo podría mejorarse el agente utilizando información adicional o fuentes externas?

Recomendación para nivel maestría

Un análisis más profundo puede incluir el diseño de agentes más complejos que integren múltiples herramientas externas como bases de datos, sistemas de búsqueda o APIs externas. También se puede estudiar el uso de estrategias más avanzadas de diseño de prompts como *chain-of-thought*, descomposición de tareas o planificación de múltiples pasos.

Otro análisis relevante consiste en evaluar la confiabilidad de las respuestas generadas por el modelo mediante diferentes estrategias de validación, así como estudiar posibles errores o alucinaciones del modelo.

También se puede analizar el uso de arquitecturas de agentes más avanzadas donde el modelo actúa como un sistema de planificación que decide cuándo utilizar herramientas externas o ejecutar funciones específicas.

Finalmente, se puede estudiar la integración de estos agentes dentro de sistemas más amplios como asistentes conversacionales, motores de búsqueda semánticos o sistemas de análisis automático de documentos.

Con esto se pueden alcanzar proyectos tipo *Stanford NLP*, *MIT AI Systems* y programas de investigación en agentes inteligentes basados en modelos de lenguaje.

Notas finales

En estos proyectos finales se demuestra el conocimiento adquirido durante el curso y se evalúa el dominio de las herramientas aprendidas para aplicarlas a casos reales y prácticos. Esto te permitirá comenzar a estructurar un portafolio que puedas presentar en entrevistas de trabajo en empresas y consultorías especializadas que requieran analistas de datos, científicos de datos, ingenieros de datos o consultores de inteligencia artificial.

Para fortalecer tu portafolio, te recomiendo preparar una presentación breve que resuma tu proyecto. Si te es posible, intenta completar cada proyecto utilizando al menos dos conjuntos de datos distintos. Todo el material generado en cada proyecto puedes organizarlo en un repositorio de Github incluyendo la presentación, el código (aunque sea parcial) y un resumen del proyecto en el archivo `README.md`, acompañado de imágenes o fragmentos de resultados relevantes.

Esto te permitirá construir un portafolio realista que muestre lo que sabes hacer. Además, puede aumentar el interés de los entrevistadores, quienes muchas veces no poseen un conocimiento técnico profundo sobre estos perfiles, pero sí son capaces de detectar potencial y observar la capacidad de enfrentar problemas complejos de forma estructurada y creativa.

Siempre que sea posible, intenta utilizar conjuntos de datos personales o de tu propia empresa, censurando cualquier información sensible, y enfoca los problemas hacia tu propia área de trabajo o interés profesional.

Te deseo un buen proyecto y mucho éxito futuro en tu trayecto :D